

Spring Data JPA Query Methods & O/R Mapping

Employee.java

```
package com.example.demo.entity;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Double salary;
    private LocalDate joiningDate;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "department_id")
    private Department department;

    // Getters and Setters
}
```

Department.java

```
package com.example.demo.entity;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String departmentName;

    @OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
    private List<Employee> employees;

    // Getters and Setters
}
```

Student.java

```
package com.example.demo.entity;

import jakarta.persistence.*;
```

```

import java.util.List;

@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private List<Course> courses;
}

```

Course.java

```

package com.example.demo.entity;

import jakarta.persistence.*;
import java.util.List;

@Entity
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String courseName;

    @ManyToMany(mappedBy = "courses")
    private List<Student> students;
}

```

EmployeeRepository.java

```

package com.example.demo.repository;

import com.example.demo.entity.Employee;
import org.springframework.data.jpa.repository.JpaRepository;
import java.time.LocalDate;
import java.util.List;

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByNameContaining(String text);

    List<Employee> findByNameStartingWith(String prefix);

    List<Employee> findByJoiningDateBetween(LocalDate start, LocalDate end);
}

```

```

        List<Employee> findBySalaryGreaterThan(Double salary);

        List<Employee> findBySalaryLessThan(Double salary);

        List<Employee> findTop5ByOrderBySalaryDesc();

        List<Employee> findAllByOrderByNameAsc();
    }

```

application.properties

```

spring.datasource.url=jdbc:mysql://localhost:3306/companydb
spring.datasource.username=root
spring.datasource.password=password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

SampleController.java

```

package com.example.demo.controller;

import com.example.demo.entity.Employee;
import com.example.demo.repository.EmployeeRepository;
import org.springframework.web.bind.annotation.*;
import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeRepository repository;

    public EmployeeController(EmployeeRepository repository) {
        this.repository = repository;
    }

    @GetMapping("/contains/{text}")
    public List<Employee> contains(@PathVariable String text) {
        return repository.findByNameContaining(text);
    }

    @GetMapping("/starts/{text}")
    public List<Employee> starts(@PathVariable String text) {
        return repository.findByNameStartingWith(text);
    }

    @GetMapping("/top")
    public List<Employee> top() {
        return repository.findTop5ByOrderBySalaryDesc();
    }

    @GetMapping("/greater/{salary}")

```

```
public List<Employee> greater(@PathVariable Double salary) {
    return repository.findBySalaryGreaterThan(salary);
}

@GetMapping("/less/{salary}")
public List<Employee> less(@PathVariable Double salary) {
    return repository.findBySalaryLessThan(salary);
}

@GetMapping("/between")
public List<Employee> between(@RequestParam LocalDate start,
                             @RequestParam LocalDate end) {
    return repository.findByJoiningDateBetween(start, end);
}

@GetMapping("/sorted")
public List<Employee> sorted() {
    return repository.findAllByOrderByNameAsc();
}
}
```